Solveur de cinématique inverse redondante
Technologie GRAM

Formulation mathématique et description algorithmique

Robot ABB IRB 6640 sur rail linéaire + mandrin rotatif — RobotComponents / Grasshopper



Rapport de stage — Annexe technique –Adrien chaudron

GRADEL — Automatisation de l'enroulement filamentaire

Sommaire

Sommaire 2
1. Position du problème 4
1.1 Contexte 4
1.2 Paramétrage cinématique 4
1.3 Formulation 4
2. Vocabulaire : cinématique directe et inverse 5
2.1 Espace articulaire et espace opérationnel 5
2.2 Cinématique directe (FK, Forward Kinematics) 5
2.3 Cinématique inverse (IK, Inverse Kinematics) 5
2.3.1 Pourquoi huit solutions ? 6
2.3.2 Résolution 6
2.4 Butées et enveloppe de travail 7
2.5 Singularités 7
2.6 Redondance 8
2.7 Glossaire 8
3. Notations et conventions 9
3.1 Repères et transformations homogènes 9
3.2 Convention de quaternion 9
3.2.1 Extraction inverse (matrice → quaternion) 10
3.3 Table des symboles 10
4. Cinématique du mandrin 11
4.1 Axe de rotation 11
4.2 Rotation autour d'un axe quelconque 11
4.3 Composition du repère utilisateur 11
5. Chaîne de mise en forme de l'orientation 13
5.1 Stabilisation de l'hémisphère 13
5.2 Rotation de yaw 13
5.3 Retournement pour le robot 13
6. Placement déterministe des axes externes 15
6.1 Position du rail : interpolation du déport latéral 15
6.1.1 Ordre de préférence sur le rail 15
6.2 Balayage en spirale 16
6.3 Yaw idéal : angle signé vers la base du robot 16
6.3.1 Ordre de préférence sur le yaw 16
7. Angle de présentation du mandrin (theta) 17
7.1 Critère d'alignement 17
7.2 Résolution retenue : balayage discret 17
7.3 Solution analytique 18
7.3.1 Séparer ce qui tourne de ce qui ne tourne pas 18

7.3.2 Projeter, puis réduire à un seul cosinus 18
7.3.3 Critère d'atteignabilité de l'alignement 19
7.4 Génération des candidats de mandrin 19
8. Algorithme de recherche 20
8.1 Prédicat d'acceptation 20
8.2 Structure des boucles 21
8.3 Court-circuit « hors d'atteinte » 21
8.4 Complexité 21
8.5 Solution de repli 22
9. Détection de collision outil / boîtier 23
9.1 Repositionnement automatique entre robots 23
9.2 Recalage rigide par paire de maillages 23
9.3 Test d'interférence en deux phases 24
10. Interface du composant 24
10.1 Principales entrées 24
10.2 Sorties de diagnostic 25
11. De la pose au robtarget : finalité du solveur 26
11.1 Structure du robtarget 26
11.2 Extraction des sept nombres 26
11.3 Le rôle du workobject 27
11.4 Condition de cohérence 27
11.5 Ce que le robtarget ne contient pas 27
11.6 Place dans la chaîne de production 28
11.6.1 Les blocs logiques de la définition Grasshopper 28
11.6.2 En amont — construction des cibles autour d'un bushing (déjà implanté auparavant) 29
11.6.3 En aval — la syntaxe d'enroulement 29
11.6.4 Séparation données / logique 29
12. Discussion et perspectives 31
12.1 Nature de l'optimum obtenu 31
12.2 Vers une optimisation globale 31
12.3 Améliorations ponctuelles identifiées 31
12.4 Synthèse 31

1. Position du problème

1.1 Contexte

L'objectif du poste de travail est de déposer un fil de carbone autour d'une série de **bushings** (douilles) fixées sur une pièce montée sur un **mandrin rotatif**. Le dépôt est réalisé par un robot ABB IRB 6640 monté sur un **rail linéaire**. La géométrie du parcours est produite en amont sous forme d'une liste de **plans cibles** : chaque plan porte l'origine du point de passage de l'outil et l'orientation souhaitée de celui-ci.

Le rôle du solveur — le composant C# `IK SOLVER` — est de transformer chacun de ces plans en une configuration réellement exécutable par le contrôleur ABB, c'est-à-dire un *robtarget* complet : six angles articulaires $A1…A6$, une position de rail et une position de mandrin.

1.2 Paramétrage cinématique

Le système comporte huit degrés de liberté commandés :

- $q = (q_1, \dots, q_6) \in \mathbb{R}^6$: les six axes du bras (A1 à A6) ;
- $E_1 \in \mathbb{R}$: la position du chariot sur le rail (axe externe `EJa`, en mm) ;
- $\theta_b \in \mathbb{R}$: l'angle du mandrin (axe externe `EJb`, en degrés).

À cela s'ajoute un degré de liberté purement géométrique : la rotation ψ de l'outil autour de la normale de la cible (le **yaw**). Comme le procédé d'enroulement est invariant par rotation de l'outil autour de son axe de dépose, ψ est un paramètre libre que le solveur peut choisir.

On dispose donc de **9 paramètres** pour satisfaire une tâche de pose qui n'en contraint que **6** (3 en position, 3 en orientation), dont un est lui-même relâché. La redondance effective est de trois dimensions :

$$dim(\mathcal{S}) = 9 - 6 + 1 = 3 \quad (1)$$

où $\mathcal{S}$ désigne la variété des solutions d'une cible donnée. C'est cette redondance qui impose une **recherche** plutôt qu'un calcul direct : il n'existe pas une solution, mais une famille continue de solutions, dont seule une partie respecte les butées articulaires et évite les collisions.

1.3 Formulation

Notons P_i le i-ième plan cible exprimé dans le repère de la pièce, $\mathcal{T}(P_i, \theta_b)$ son image dans le repère monde lorsque le mandrin est à l'angle θ_b, et Φ la chaîne de mise en forme d'orientation décrite au §5. Le problème résolu pour chaque cible est :

$$\text{Trouver } (E_1, \theta_b, \psi, q) \text{ tel que } FK(q, E_1, \theta_b) = \Phi(\mathcal{T}(P_i, \theta_b), \psi) \quad (2)$$

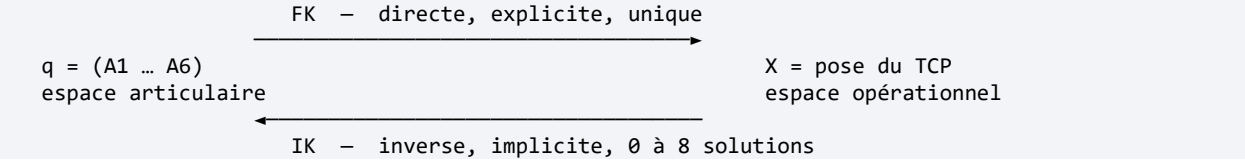
$$\text{sous } q \in \mathcal{Q} = \prod_{k=1}^{6} [\, q_k\text{^min} , q_k\text{^max} \,], \quad E_1 \in [E_1\text{^min} , E_1\text{^max}], \quad Collision(q, E_1, \theta_b) = faux \quad (3)$$

où FK est le modèle géométrique direct du robot (position du TCP dans le monde à partir des articulations et des axes externes). Le solveur ne cherche pas le **meilleur** quadruplet mais le **premier** quadruplet admissible rencontré selon un ordre de parcours soigneusement construit : ce point est développé au §8.

> **Pourquoi une recherche et pas une optimisation ?**
> Chaque évaluation d'un candidat coûte un appel de cinématique inverse et, potentiellement, un calcul de collision maillage-maillage. Une optimisation globale sur les 3 dimensions de redondance, pour chacune des quelques centaines de cibles d'un cycle, serait prohibitive dans l'environnement Grasshopper. Le choix retenu est un parcours **ordonné par préférence décroissante** : le premier candidat valide est, par construction de l'ordre, proche du candidat idéal.

2. Vocabulaire : cinématique directe et inverse

Deux opérations reviennent en permanence dans la suite : la **cinématique directe** et la **cinématique inverse**. Elles relient les deux façons de décrire l'état d'un robot — par ses **articulations** d'une part, par la **pose de son outil** d'autre part — et sont réciproques l'une de l'autre, mais absolument pas symétriques en difficulté.



2.1 Espace articulaire et espace opérationnel

L'**espace articulaire** est celui des commandes réelles envoyées au contrôleur : les six angles $q = (q_1, \ldots, q_6)$ des moteurs, notés $A1$ à $A6$ dans le vocabulaire ABB, auxquels s'ajoutent ici les deux axes externes E_1 (rail) et θ_b (mandrin).

L'**espace opérationnel** est celui de la tâche : la pose $X \in SE(3)$ du point caractéristique de l'outil — le **TCP** (*Tool Center Point*) — soit trois coordonnées de position et trois d'orientation. C'est dans cet espace que la géométrie d'enroulement est définie : les plans cibles du §1 sont des poses.

Le problème du solveur naît de ce que la géométrie s'exprime naturellement dans l'espace opérationnel, alors que le robot ne s'exécute que dans l'espace articulaire.

2.2 Cinématique directe (FK, Forward Kinematics)

La cinématique directe répond à la question : *« où se trouve l'outil si je place les moteurs à telle valeur ? »* Elle se calcule en composant les transformations de chaque segment du bras :

$$X = FK(q) = T_base \cdot A_1(q_1) \cdot A_2(q_2) \cdot A_3(q_3) \cdot A_4(q_4) \cdot A_5(q_5) \cdot A_6(q_6) \cdot T_outil \quad (4)$$

où chaque $A_k(q_k)$ est la matrice homogène 4 × 4 du §3.1 décrivant le passage du segment $k{-}1$ au segment k. Dans la convention de **Denavit-Hartenberg**, quatre paramètres suffisent à la décrire :

$$A_k(q_k) = Rot_z(q_k + \theta_k) \cdot Trans_z(d_k) \cdot Trans_x(a_k) \cdot Rot_x(\alpha_k) \quad (5)$$

les trois constantes d_k, a_k, α_k étant des dimensions mécaniques figées du robot (longueurs de segments et angles entre axes), et q_k la seule variable. Le modèle du IRB 6640 chargé dans RobotComponents encapsule ces constantes ; c'est pourquoi le solveur n'a jamais à les manipuler explicitement.

Avec le rail, la base elle-même se déplace, ce qui ajoute simplement un facteur en tête de la chaîne :

$$T_base(E_1) = Trans(E_1 \cdot u_rail) \cdot T_base(0) \quad (6)$$

La FK a trois propriétés remarquables : elle est **explicite** (un simple produit de matrices), **toujours définie** (toute combinaison d'angles correspond à une pose), et **unique** (une configuration articulaire donne exactement une pose). Elle est de surcroît peu coûteuse.

Dans le solveur, la FK sert à trois usages : déterminer la pose du TCP au repos pour le calcul de *theta* (§7.1), positionner les maillages du robot pour le test de collision (§9), et produire l'affichage de contrôle dans Grasshopper.

2.3 Cinématique inverse (IK, Inverse Kinematics)

La cinématique inverse répond à la question réciproque : *« quelles valeurs donner aux moteurs pour que l'outil se trouve à telle pose ? »* Formellement, il ne s'agit plus d'une fonction mais d'un **ensemble solution** :

$$IK(X) = \{\, q \in \mathbb{R}^6 : FK(q) = X \,\} \quad (7)$$

et cet ensemble peut être vide, fini, ou infini. C'est là toute la différence avec la FK :

- **Vide** : la pose demandée est hors de l'enveloppe de travail — trop loin, trop près, ou dans une orientation inatteignable. C'est le diagnostic `out of reach` exploité au §8.3.
- **Fini** : cas général. Pour un robot à six axes rotoïdes muni d'un **poignet sphérique** — les axes 4, 5 et 6 concourants, ce qui est le cas de l'IRB 6640 — il existe jusqu'à **huit** solutions distinctes.
- **Infini** : au voisinage d'une singularité (§2.5), ou lorsque le système est redondant, comme ici avec les axes externes.

2.3.1 Pourquoi huit solutions ?

Le poignet sphérique permet de **découpler** le problème en deux sous-problèmes de dimension 3 : les trois premiers axes placent le **centre du poignet**, les trois derniers l'orientent. Chaque sous-problème admet des choix binaires :

Choix	Description	Notation ABB
Épaule	le bras atteint la cible par la gauche ou par la droite de l'axe 1	cf1
Coude	le coude pointe vers le haut ou vers le bas	cf4
Poignet	l'orientation est atteinte directement ou par retournement (A4 et A6 à +180°, A5 de signe opposé)	cf6

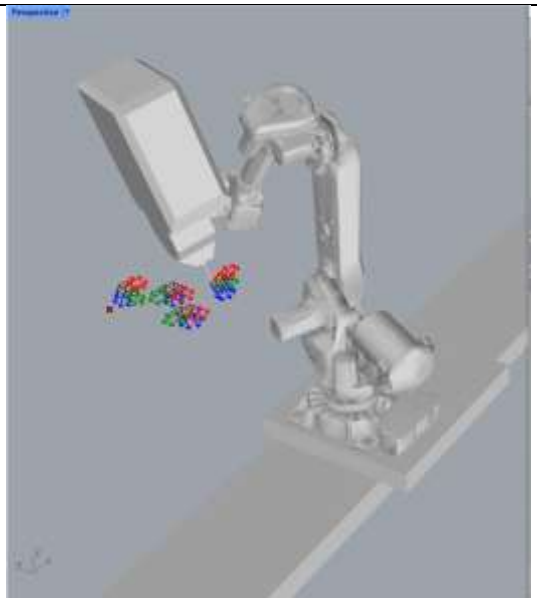
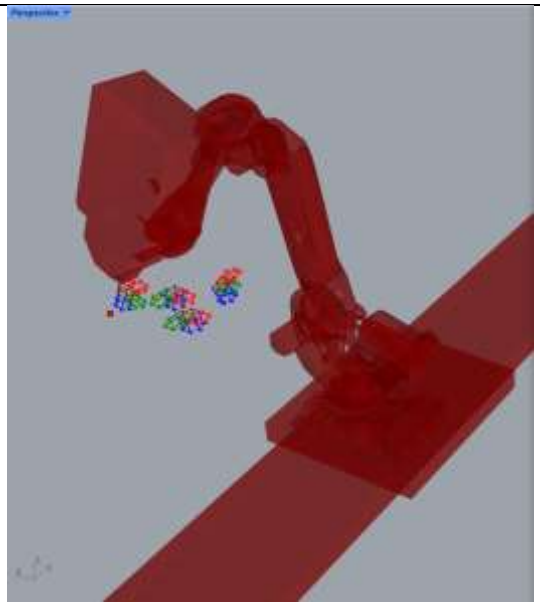
soit $2 \times 2 \times 2 = 8$ combinaisons. Le contrôleur ABB désigne celle qu'il doit utiliser par une donnée de configuration, le **confdata**, noté *[cf1, cf4, cf6, cfx]*. Dans le solveur, ce quadruplet est fixé à `ConfigurationData(1, 0, 0, 0)` : on impose donc une famille de posture, ce qui réduit l'ensemble solution à un unique élément et rend le résultat **déterministe et reproductible** d'une exécution à l'autre.

> **Conséquence pratique**
> Fixer le confdata est un choix de conception assumé : il évite qu'un même programme se comporte différemment d'un lancement à l'autre, et garantit que deux points consécutifs du parcours n'appartiennent pas à des familles de posture différentes — ce qui provoquerait un **reconfigurage** du bras, c'est-à-dire un mouvement ample et imprévisible entre deux points pourtant voisins. Le prix à payer est qu'une pose atteignable dans une autre configuration sera déclarée insoluble par le solveur.

2.3.2 Résolution

Pour cette famille de robots, l'IK admet une **solution analytique fermée** — c'est une propriété du poignet sphérique, connue sous le nom de condition de Pieper. Il n'y a donc ni itération ni convergence à surveiller : RobotComponents applique directement les formules et retourne, outre les angles, deux verdicts que le solveur exploite :

- `ErrorText` : la liste des messages d'échec (hors d'atteinte, singularité, configuration impossible) ;
- `IsInLimits` : le respect des butées du modèle de robot chargé.

	
Robot cinématiquement cohérent	Erreur de position (singularité, limite d'axe…)

2.4 Butées et enveloppe de travail

Chaque axe est mécaniquement borné. L'ensemble des configurations physiquement admissibles est donc un **pavé** de l'espace articulaire :

$$\mathcal{Q} = \prod_{k=1}^{6} [\, q_k\text{^}min \,,\, q_k\text{^}max \,] \quad (8)$$

Son image par la FK, $FK(\mathcal{Q})$, est l'**enveloppe de travail** : l'ensemble des poses réellement atteignables. Une pose hors de cette enveloppe donne un ensemble $IK(X)$ vide. Noter que l'enveloppe n'est pas une simple sphère : elle comporte un creux central inaccessible autour de la colonne du robot, et sa forme dépend de l'outil monté.

Le solveur distingue deux niveaux de butées : celles du **modèle constructeur** (vérifiées par `IsInLimits`) et celles, plus serrées, définies par l'utilisateur dans l'entrée `joint_limits`, qui permettent d'interdire des postures légales mais indésirables.

2.5 Singularités

Une **singularité** est une configuration où le robot perd instantanément la capacité de se déplacer dans certaines directions. Elle se caractérise par la **matrice jacobienne** $J(q) = \partial FK / \partial q$, qui relie vitesses articulaires et vitesse de l'outil :

$$\dot{X} = J(q) \cdot \dot{q} \quad \Longrightarrow \quad \dot{q} = J(q)^{-1} \cdot \dot{X} \quad (9)$$

Lorsque $\det J(q) = 0$, la jacobienne n'est plus inversible : certaines directions de l'espace opérationnel deviennent inatteignables, et surtout, au **voisinage** de cette configuration, les vitesses articulaires nécessaires **divergent** — le robot s'emballe ou déclenche un arrêt d'urgence, alors même que la trajectoire demandée reste lente. C'est pourquoi une singularité doit être évitée dans un rayon confortable, et pas seulement en son point exact.

On distingue classiquement trois familles :

- **Singularité de poignet** : $A5 = 0$, les axes 4 et 6 deviennent colinéaires et se neutralisent. C'est la plus fréquente et la raison pour laquelle les butées de projet excluent souvent une bande autour de $A5 = 0$.
- **Singularité d'épaule** : le centre du poignet se retrouve sur la verticale de l'axe 1, au-dessus de la base ; l'angle $A1$ n'est plus défini.

- **Singularité de coude** : le bras est complètement tendu, le centre du poignet atteint la limite d'enveloppe.

> **Attention à la terminologie ABB**
>
> Le message d'erreur « `elbow singularity` » renvoyé par la chaîne ABB ne désigne **pas** une hyperextension du coude, comme le nom le suggère, mais le cas où le **centre du poignet approche la verticale de l'axe 1** au-dessus de la base du robot. Le remède n'est donc pas de replier le bras mais de **déplacer le centre du poignet latéralement**, ce qui se fait ici en jouant sur le rail (`EJa`) ou sur l'angle du mandrin (`EJb`). C'est précisément ce que font les balayages de secours du §6.1.

Dans le solveur, les singularités ne sont pas détectées par un calcul de jacobienne : elles remontent sous forme de messages d'erreur de l'IK, regroupés avec les cas hors d'atteinte dans le prédicat `IsOutOfReach` (§8.3).

2.6 Redondance

Un système est dit **redondant** lorsqu'il possède plus de degrés de liberté que la tâche n'en contraint. Avec ses six axes, le bras seul n'est pas redondant pour une tâche de pose : à confdata fixé, l'IK admet une solution unique — ou aucune.

L'ajout du rail, du mandrin et du yaw libre change la nature du problème : l'ensemble IK devient une **variété continue** de dimension 3, et il n'y a plus de solution à « calculer » mais une famille à **explorer**. Toute la structure du solveur découle de ce constat.

2.7 Glossaire

Terme	Définition
FK — cinématique directe	articulations → pose de l'outil ; explicite, unique, peu coûteuse
IK — cinématique inverse	pose de l'outil → articulations ; 0, 8 ou une infinité de solutions
TCP — Tool Center Point	point caractéristique de l'outil dont on commande la pose
tool0	repère de la bride du robot, avant montage de l'outil ; le TCP en est un décalage fixe
Pose	position + orientation, soit un élément de SE(3) ; représentée par un repère ou une matrice 4 × 4
robtarget	instruction ABB décrivant une cible : position, quaternion, confdata, axes externes
Workobject (wobj)	repère de référence dans lequel une cible est exprimée ; se décompose en uframe (mobile) et oframe (pièce)
uframe / oframe	repère porteur (ici entraîné par le mandrin) et repère de la pièce sur ce porteur
Axe externe	axe motorisé hors du bras : EJa pour le rail, EJb pour le mandrin
confdata	quadruplet [cf1, cf4, cf6, cfx] désignant la famille de posture à retenir parmi les huit
Butée articulaire	borne mécanique d'un axe ; leur produit forme le pavé Q
Enveloppe de travail	image du pavé des butées par la FK ; ensemble des poses atteignables
Singularité	configuration où la jacobienne perd son rang ; les vitesses articulaires y divergent
Redondance	excédent de degrés de liberté sur la tâche ; donne une famille continue de solutions
MoveL / MoveJ	déplacement interpolé en ligne droite dans l'espace opérationnel / interpolé dans l'espace articulaire
RAPID	langage de programmation des contrôleurs ABB, cible finale de la chaîne

3. Notations et conventions

3.1 Repères et transformations homogènes

Un **repère** (objet `Plane` de Rhino) est un quadruplet $P = (O, x, y, z)$ formé d'une origine et d'une base orthonormée directe, avec :

$$z = x \times y\,, \quad \|x\| = \|y\| = \|z\| = 1\,, \quad x \cdot y = y \cdot z = z \cdot x = 0 \tag{10}$$

On lui associe une matrice $T \in SE(3)$ de taille **4 × 4**. En notant $x = (x_1, x_2, x_3)$, $y = (y_1, y_2, y_3)$, $z = (z_1, z_2, z_3)$ les composantes des trois axes et $O = (O_1, O_2, O_3)$ celles de l'origine, elle s'écrit sous forme développée :

$$T = \begin{bmatrix} x1 & y1 & z1 & O1 \\ x2 & y2 & z2 & O2 \\ x3 & y3 & z3 & O3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Chacun des trois axes occupe donc une **colonne entière**, et non une seule case. On résume cette structure par la notation en blocs :

$$T = \begin{bmatrix} R & O \\ 0 & 1 \end{bmatrix} \qquad \begin{matrix} R = [\ x\ \ y\ \ z\ ] \\ O \\ [\ 0\ \ 0\ \ 0\ \ 1\ ] \end{matrix} \qquad \begin{matrix} \text{bloc 3x3 , rotation} \quad (\ R \in SO(3)\ ) \\ \text{bloc 3x1 , translation} \\ \text{ligne de complement} \end{matrix}$$

La quatrième ligne et la quatrième colonne n'ont pas de contenu géométrique propre : elles servent à **regrouper rotation et translation en une seule multiplication**. Une matrice 3 × 3 seule ne le permettrait pas, puisque $v \mapsto R\,v$ laisse toujours l'origine fixe. On complète donc le point d'une quatrième coordonnée valant 1 :

$$\begin{bmatrix} R & O \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ 1 \end{bmatrix} = \begin{bmatrix} R\ p + O \\ 1 \end{bmatrix} \qquad \text{soit} \quad p' = R\ p + O$$

L'intérêt est double : la composition de deux changements de repère se réduit à un produit matriciel — ce dont on se sert au §4.3 — et l'inverse d'un changement de repère est simplement l'inverse de la matrice.

Un **vecteur** (une direction, non un point) reçoit au contraire un 0 en quatrième coordonnée, ce qui élimine la translation du produit. D'où la distinction fondamentale :

$$p' = R\,p + O \ \ \text{(point)}\,, \qquad v' = R\,v \ \ \text{(vecteur)} \tag{11}$$

Cette distinction est explicitement codée dans `TransformPlaneWobj` : l'origine du plan subit T complète, tandis que ses axes subissent T privée de sa colonne de translation (mise à zéro de `M03`, `M13`, `M23`). L'oublier fait « dériver » les axes du plan avec la translation et produit des orientations fausses dès que le workobject n'est pas à l'origine du monde.

3.2 Convention de quaternion

Les poses fournies par le contrôleur ABB (chaînes `iAxisStr` et `iOframeStr`) sont au format :

```
"px, py, pz, qw, qx, qy, qz"
```

soit un quaternion **scalaire en tête** — convention ABB, à ne pas confondre avec la convention (x, y, z, w) de nombreuses bibliothèques. Le quaternion est renormalisé à la lecture :

$$q \leftarrow q \,/\, \|q\|\,, \quad \|q\| = \sqrt{(w^2 + x^2 + y^2 + z^2)} \tag{12}$$

La matrice de rotation associée s'écrit :

```
R(q) = ⎡ 1-2yy-2zz    2xy-2zw     2xz+2yw   ⎤
       ⎢ 2xy+2zw     1-2xx-2zz    2yz-2xw   ⎥
       ⎣ 2xz-2yw     2yz+2xw     1-2xx-2yy  ⎦
```

3.2.1 Extraction inverse (matrice → quaternion)

La fonction `RotmatToQuat` implémente la méthode de **Shepperd**, qui choisit parmi quatre branches celle dont le dénominateur est le plus grand, afin d'éviter une division par une quantité proche de zéro. En notant $tr(R) = R_{11} + R_{22} + R_{33}$, la branche principale ($tr > 0$) est :

$$s = 2\sqrt{(tr(R) + 1)}\,,\quad w = s/4\,,\quad x = (R_{32} - R_{23})/s\,,\quad y = (R_{13} - R_{31})/s\,,\quad z = (R_{21} - R_{12})/s \tag{13}$$

Sinon on bascule sur la branche associée au plus grand élément diagonal. Par exemple si R_{11} domine :

$$s = 2\sqrt{(1 + R_{11} - R_{22} - R_{33})}\,,\quad x = s/4\,,\quad w = (R_{32} - R_{23})/s\,,\quad y = (R_{12} + R_{21})/s\,,\quad z = (R_{13} + R_{31})/s \tag{14}$$

Les deux autres branches s'obtiennent par permutation circulaire des indices. Le quaternion est renormalisé en sortie.

3.3 Table des symboles

Symbole	Signification	Origine dans le code
$q = (q_1 \ldots q_6)$	angles articulaires A1 à A6 (degrés)	RobotJointPosition
E_1	position du chariot sur le rail (mm)	EJa / ext[0]
θ_b	angle du mandrin (degrés)	EJb / ext[1]
θ_teach	angle mandrin de référence à l'apprentissage	iTeachingAngle
ψ	rotation de l'outil autour de la normale (yaw)	yaw
a_0, a	point et direction de l'axe du mandrin	axisOrigin, axisDir
T_a, T_o	pose de l'axe et de l'oframe (wobj ABB)	iAxisStr, iOframeStr
$T_u(\theta_b)$	pose du repère utilisateur à l'angle θ_b	ComputeUframePose
T_ref	T_u évaluée à θ_teach (référence)	Tref
n_0	normale brute de la bushing en monde	nRaw
z_t	axe Z du TCP à la pose de repos	tcpHome.ZAxis
$\mathcal{Q}$	pavé des butées articulaires	joint_limits

4. Cinématique du mandrin

Le mandrin est modélisé exactement comme le contrôleur ABB modélise un **workobject sur axe externe** : un repère *oframe* solidaire de la pièce, porté par un repère *uframe* qui tourne autour d'un axe fixe. Reproduire cette composition à l'identique dans Grasshopper est la condition pour que le code RAPID généré soit interprété de la même manière par le robot réel.

4.1 Axe de rotation

La chaîne `iAxisStr` définit $T_a = [\, R(q_a) \,|\, p_a \,]$. L'axe de rotation est la droite passant par $a_0 = p_a$ et dirigée par la k-ième colonne de $R(q_a)$, où k est fixé par le paramètre `iMandrelAxis` (« `z` » en pratique) :

$$a = R(q_a) \cdot e_k \,, \quad k \in \{x \to 1, y \to 2, z \to 3\} \,, \quad \|a\| = 1 \tag{15}$$

4.2 Rotation autour d'un axe quelconque

La rotation d'angle δ autour du vecteur unitaire $a = (a_x, a_y, a_z)$ est donnée par la formule de **Rodrigues** sous forme matricielle :

$$R(a, \delta) = I \cos\delta + (1 - \cos\delta)\, a\, a^{T} + \sin\delta\, [a]_{\times} \tag{16}$$

où $[a]_{\times}$ est la matrice antisymétrique du produit vectoriel. En notant $a = (x, y, z)$ les composantes de l'axe et en posant $c = \cos\delta$, $s = \sin\delta$, $C = 1 - c$, c'est exactement la matrice codée dans `AxisAngleRot` :

```
R(a, d) = [ x x C + c     x y C - z s     x z C + y s ]
          [ y x C + z s   y y C + c       y z C - x s ]
          [ z x C - y s   z y C + x s     z z C + c   ]
```

La rotation n'ayant pas lieu autour de l'origine du monde mais autour d'une droite passant par a_0, on conjugue par la translation correspondante. La transformation affine complète, implémentée par `RotationAboutAxis`, est :

$$T_rot(a_0, a, \delta) = Trans(a_0) \cdot Rot(a, \delta) \cdot Trans(-a_0) = [\, R(a,\delta) \,|\, a_0 - R(a,\delta)\, a_0 \,] \tag{17}$$

On vérifie immédiatement que a_0 est bien un point fixe : $R\, a_0 + a_0 - R\, a_0 = a_0$.

4.3 Composition du repère utilisateur

L'angle réellement appliqué est un angle **relatif** à l'apprentissage, car la géométrie a été numérisée alors que le mandrin se trouvait à θ_teach :

$$\delta(\theta_b) = (\theta_b - \theta_teach) \cdot \pi / 180 \tag{18}$$

La pose du repère de la pièce dans le monde est alors la composition :

$$T_u(\theta_b) = T_rot(a_0, a, \delta(\theta_b)) \cdot T_a \cdot T_o \tag{19}$$

Cette composition doit être lue de droite à gauche : on part de l'*oframe* exprimé dans le repère de l'axe, on le remonte dans le monde par T_a, puis on applique la rotation du mandrin. Toute permutation de ces trois facteurs produit un décalage silencieux qui ne se manifeste que pour certaines positions du workobject — c'est un piège classique de cette chaîne.

La transformation de référence, utilisée pour ramener les plans de la pièce dans le monde à l'apprentissage, est le cas particulier $\delta = 0$:

$$T_ref = T_u(\theta_teach) = T_a \cdot T_o \tag{20}$$

Enfin, la pose T_u est reconvertie en couple *(position, quaternion)* pour être écrite dans la chaîne du workobject RAPID, en respectant la convention scalaire en tête du §3.2.

Validation numérique

Cette composition a été validée à 0,000 mm contre la chaîne de workobject produite indépendamment par le contrôleur : les deux poses coïncident pour tout angle de mandrin. La validation par valeurs numériques — et non par inspection visuelle — est indispensable ici, car une erreur de composition peut se compenser pour une position particulière du workobject tout en étant fausse en général.

5. Chaîne de mise en forme de l'orientation

Un plan issu de la modélisation géométrique n'est pas directement utilisable comme cible robot : son orientation est arbitraire à trois égards — le sens de la normale n'est pas garanti, l'axe x n'a pas de signification procédé, et la convention robot veut que l'axe z de l'outil **pointe vers la matière**. La chaîne appliquée est :

$$\Phi = FixPlaneForRobot \circ RotatePlane(\psi) \circ StabilizePlane$$

5.1 Stabilisation de l'hémisphère

La première étape lève l'ambiguïté de signe de la normale en la forçant dans l'hémisphère supérieur du monde, puis réorthonormalise la base par **procédé de Gram-Schmidt** :

$$\tilde{z} = sgn(z \cdot e_z) \cdot z / \|z\| \quad (22)$$

$$\tilde{x} = (x - (x \cdot \tilde{z})\,\tilde{z}) / \|x - (x \cdot \tilde{z})\,\tilde{z}\|, \quad \tilde{y} = \tilde{z} \times \tilde{x} \quad (23)$$

Le terme $(x \cdot \tilde{z})\,\tilde{z}$ est la projection orthogonale de x sur la normale ; la retrancher garantit $\tilde{x} \perp \tilde{z}$ même si le plan d'entrée est légèrement non orthogonal. Un cas dégénéré subsiste : si x est colinéaire à $\tilde{z}$, la projection annule le vecteur. Le code bascule alors sur un vecteur de secours :

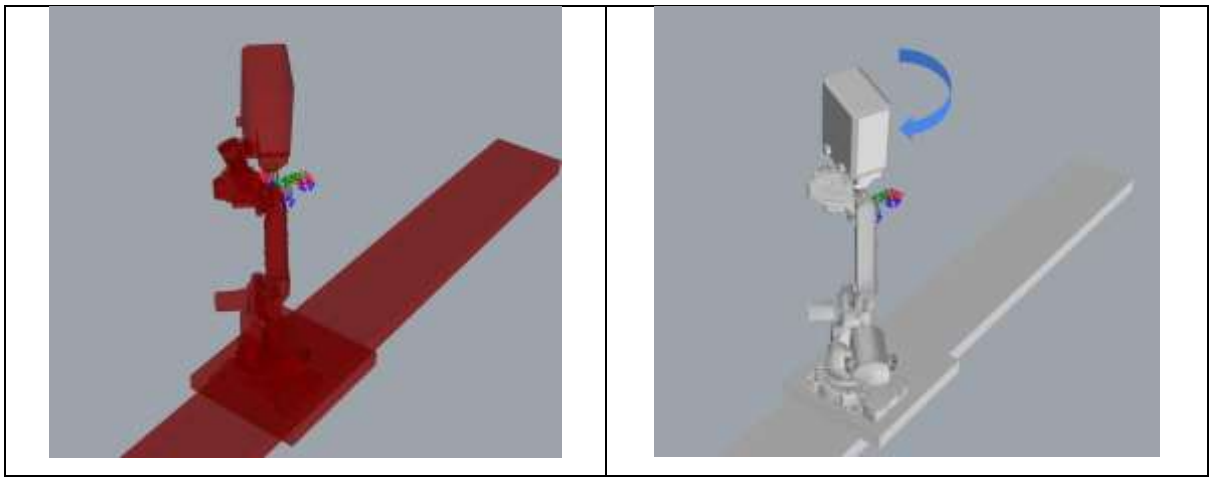
$$\tilde{x} \leftarrow (e_x - (e_x \cdot \tilde{z})\,\tilde{z}) / \|\cdot\| \quad si \ \|x - (x \cdot \tilde{z})\,\tilde{z}\| \approx 0 \quad (24)$$

5.2 Rotation de yaw

La rotation d'angle ψ autour de la normale **propre** du plan, passant par son origine, laisse la position et la normale inchangées et ne fait tourner que le repère tangent :

$$x_\psi = \cos\psi \cdot \tilde{x} + \sin\psi \cdot \tilde{y}, \quad y_\psi = -\sin\psi \cdot \tilde{x} + \cos\psi \cdot \tilde{y}, \quad z_\psi = \tilde{z} \quad (25)$$

C'est précisément le degré de liberté libre du procédé : le fil est déposé le long de la normale, la rotation de l'outil autour de cette normale ne modifie pas le résultat physique mais change radicalement la posture du poignet. C'est le levier principal du solveur pour ramener $A4$, $A5$ et $A6$ dans leurs butées.



5.3 Retournement pour le robot

Enfin, la normale est inversée pour respecter la convention outil, avec une nouvelle réorthonormalisation :

$$\hat{z} = -z_\psi, \quad \hat{x} = (x_\psi - (x_\psi \cdot \hat{z})\,\hat{z}) / \|\cdot\|, \quad \hat{y} = \hat{z} \times \hat{x} \quad (26)$$

Comme x_ψ est déjà orthogonal à z_ψ — donc à $\hat{z}$ — le produit scalaire $x_\psi \cdot \hat{z}$ est nul et la projection est l'identité. Il en résulte l'identité remarquable :

$$\Phi_fix(O, x, y, z) = (O, x, -y, -z) \quad (27)$$

autrement dit **le retournement est exactement une rotation de 180° autour de l'axe x du plan**. Cette lecture est utile au débogage : si l'on constate un écart de 180° sur $A6$ entre deux versions de la définition, il faut chercher une application double ou nulle de cette étape, et non un problème de cinématique inverse.

6. Placement déterministe des axes externes

Avant toute recherche, le solveur calcule pour chaque cible un point de départ « idéal » pour le rail, pour le yaw et pour le mandrin. La qualité de ces trois estimations conditionne entièrement la performance de l'algorithme, puisque celui-ci accepte le premier candidat valide en s'éloignant progressivement de ce point de départ.

6.1 Position du rail : interpolation du déport latéral

Le robot doit se tenir à une distance latérale de la pièce — le **déport** — qui n'est pas constante : elle est interpolée linéairement le long de l'axe Y de la pièce, entre d_min à Y_min et d_max à Y_max. En notant $O = (X_O, Y_O, Z_O)$ l'origine de la cible en monde :

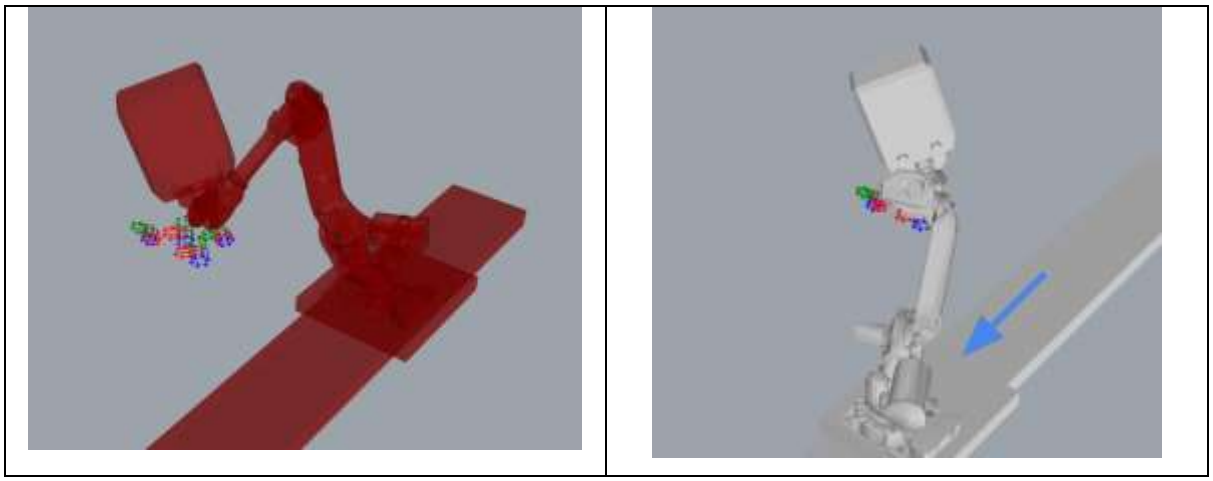
$$Y_c = min(\, max(\, Y_O \,,\, Y_min \,) \,,\, Y_max \,) \qquad (saturation) \quad (28)$$

$$\alpha = (\, Y_c - Y_min \,) \,/\, (\, Y_max - Y_min \,) \;\in [0,\, 1] \quad (29)$$

$$d(\alpha) = (1 - \alpha) \cdot d_min + \alpha \cdot d_max \quad (30)$$

$$E_1 = X_O + \varepsilon \cdot d(\alpha) \,, \qquad \varepsilon = +1 \; si \; flipOffsetDirection, \; -1 \; sinon \quad (31)$$

La saturation de Y_O garantit que α reste dans $[0,1]$ et donc que le déport reste dans l'intervalle $[d_min,\, d_max]$ même pour des cibles situées au-delà de la plage d'interpolation ; sans elle, l'extrapolation linéaire produirait des positions de rail aberrantes aux extrémités de la pièce.



Le signe ε détermine **de quel côté de la pièce le robot se place**. C'est le paramètre le plus influent de tout le solveur : du mauvais côté, le robot doit atteindre ses cibles en contournant la pièce, et la quasi-totalité d'entre elles deviennent hors d'atteinte ou en posture repliée.

La position obtenue est enfin bornée par les limites physiques du rail :

$$E_1 \in [\, 0 \,,\, 9000 \,] \;\; mm \quad (32)$$

6.1.1 Ordre de préférence sur le rail

Le rail n'est pas testé à sa seule valeur déterministe. La liste des décalages essayés, dans l'ordre, est :

1. **Continuité :** le décalage qui ramène le chariot à la position retenue pour la cible précédente, $\Delta = E_1\,\hat{}\,prev - E_1\,\hat{}\,base$. Cette priorité minimise le déplacement du rail entre deux points consécutifs, donc le temps de cycle et les à-coups.
2. **Déterministe :** $\Delta = 0$, c'est-à-dire la valeur issue de l'interpolation ci-dessus.
3. **Secours :** un balayage en spirale autour du déterministe, de pas 200 mm et d'amplitude ±1400 mm.

6.2 Balayage en spirale

Le motif de balayage utilisé aussi bien pour le rail que pour le yaw est une **spirale alternée** autour de zéro, de pas s et d'amplitude R_{max} :

$$\mathcal{S}(s, R_{max}) = (0, +s, -s, +2s, -2s, \ldots, +\lfloor R_{max}/s \rfloor \cdot s, -\lfloor R_{max}/s \rfloor \cdot s) \quad (33)$$

Ce parcours a la propriété d'énumérer les candidats par **distance croissante** au point de départ : le k-ième élément est à distance $\lceil k/2 \rceil \cdot s$. Le premier candidat accepté est donc, parmi tous les candidats valides testés, l'un des plus proches de l'idéal — c'est ce qui donne son sens à une stratégie « premier valide ».

6.3 Yaw idéal : angle signé vers la base du robot

Le yaw de départ est celui qui oriente l'axe x du plan cible vers la base du robot — posture naturellement la plus ouverte. Soit $B = (E_1, 0, 0)$ le point de base du robot sur le rail. On projette la direction base-cible dans le plan tangent de la cible :

$$D = B - O, \quad D_{\parallel} = D - (D \cdot z)\, z, \quad u = D_{\parallel} / \|D_{\parallel}\| \quad (34)$$

puis on mesure l'angle **signé** de x vers u, le signe étant donné par l'orientation du produit vectoriel par rapport à la normale :

$$\psi^* = \mathrm{sgn}(z \cdot (x \times u)) \cdot \arccos(\mathrm{clamp}(x \cdot u, -1, 1)) \quad (35)$$

Le *clamp* est indispensable : les erreurs d'arrondi peuvent produire un produit scalaire de l'ordre de $1 + 10^{-16}$, pour lequel *arccos* renvoie `NaN`.

Le cas dégénéré $\|D_{\parallel}\| \approx 0$ correspond à une cible dont la normale pointe exactement vers la base du robot ; l'angle n'est alors pas défini et le solveur retombe sur $\psi^* = 0$.

6.3.1 Ordre de préférence sur le yaw

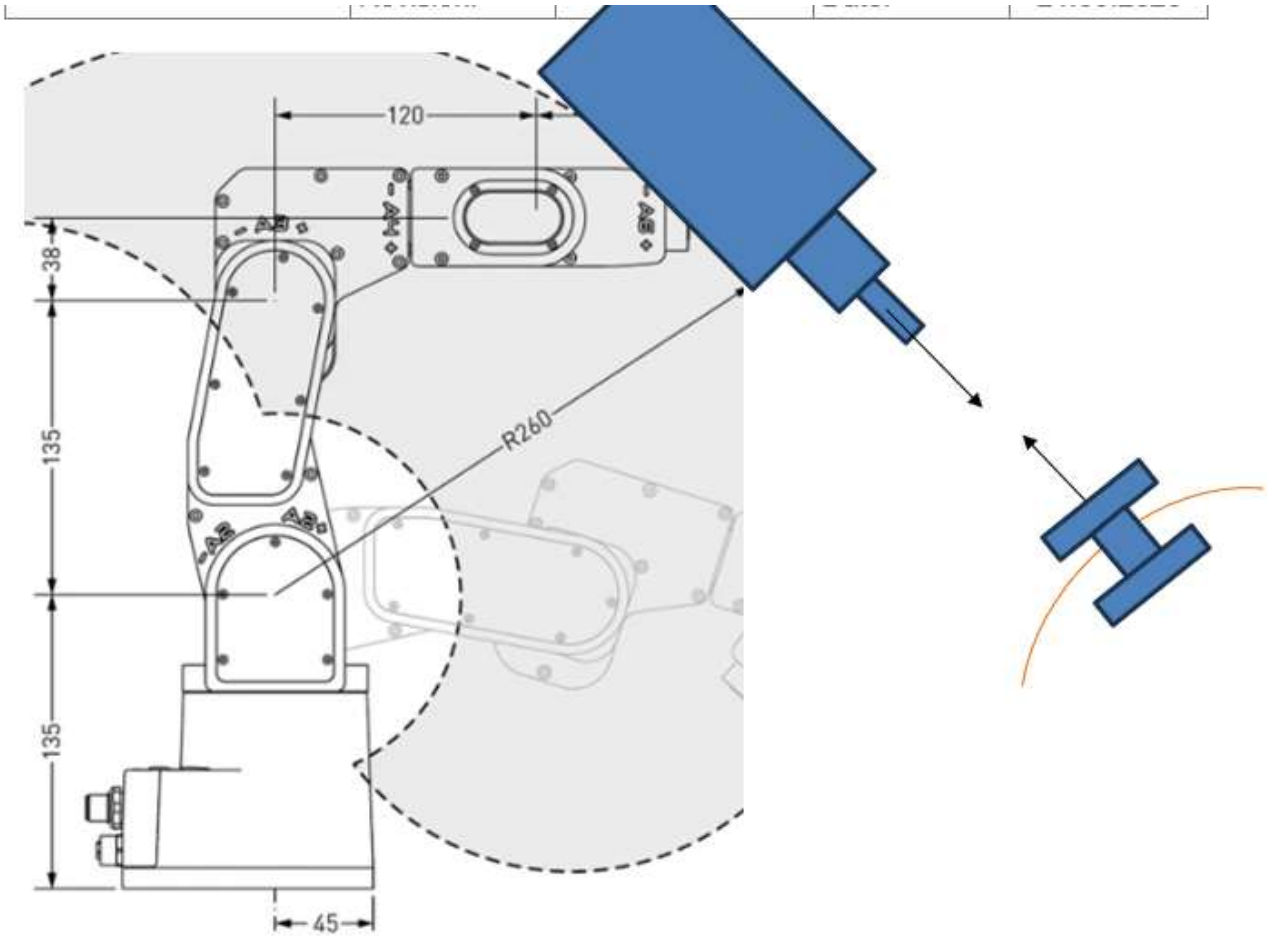
Contrairement au rail, le premier yaw essayé n'est pas ψ^* mais $\psi = 0$, obtenu par le décalage initial $\Delta\psi = -\psi^*$. Autrement dit, on tente d'abord le plan brut, sans aucune réorientation, avant de partir en spirale autour du yaw idéal :

$$\Psi = (-\psi^*) \frown \mathcal{S}(\mathrm{YAW_STEP}, \mathrm{YAW_MAX_RANGE}), \quad \psi = \psi^* + \Delta\psi \quad (36)$$

Ce choix privilégie la fidélité à l'orientation demandée par la géométrie d'enroulement : la réorientation n'intervient que si la pose brute est inaccessible. Cette méthode permet de gagner un temps de calcul considérable pour les pièces « simples » ou les points sont tous coplanaires.

7. Angle de présentation du mandrin (theta)

Lorsque le mandrin est actif et que l'option `iUseTheta` est activée, la graine de recherche θ_s n'est pas une constante mais l'angle qui **présente la bushing face à l'outil**. C'est le calcul le plus spécifique du solveur.



7.1 Critère d'alignement

Deux vecteurs sont mis en regard :

- n_0 : la normale **brute** de la bushing, ramenée en monde par la seule transformation T_ref
- z_t : l'axe z du TCP lorsque le robot est à sa **pose de repos** q_home et le rail à $E_1\,\hat{}\,base$, obtenu par cinématique directe : $z_t = FK(q_home, E_1\,\hat{}\,base).z$.

Sous rotation d'angle τ du mandrin, la normale devient $n(\tau) = R(a, \tau) \cdot n_0$. L'angle recherché est celui qui rend n et z_t **antiparallèles** de sorte à ce que le bushing « regarde » l'outil :

$$\tau^* = \arg\max_\tau \; f(\tau), \quad f(\tau) = -\left(R(a, \tau)\, n_0\right) \cdot z_t \tag{37}$$

La valeur $f(\tau^*) = +1$ correspond à un alignement antiparallèle parfait. La graine du mandrin s'obtient en repassant de l'angle **relatif** à l'angle **absolu** :

$$\theta_s = \tau^* + \theta_teach \tag{38}$$

7.2 Résolution retenue : balayage discret

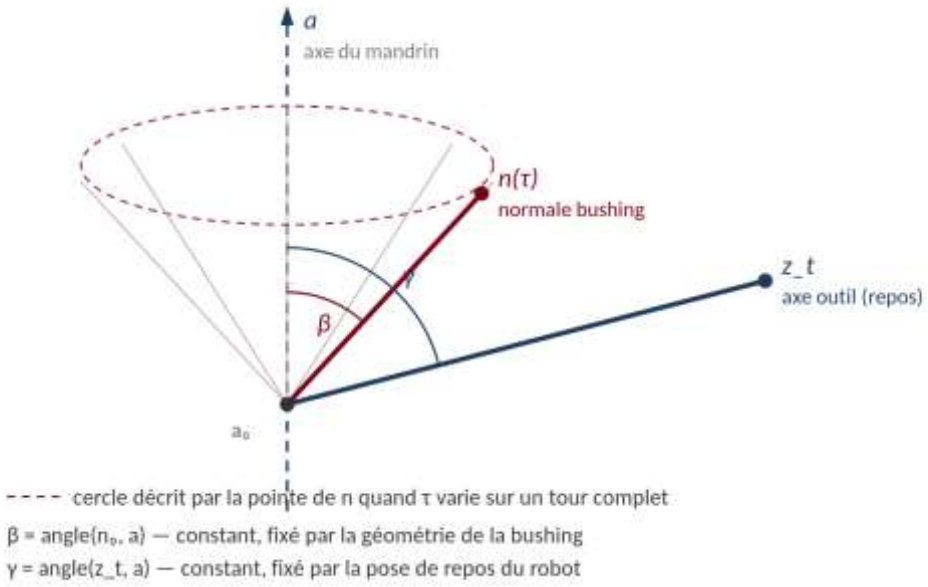
L'implémentation évalue f sur une grille régulière de 360 points :

$$\tau^* \approx \arg\max \{ f(\tau) : \tau \in \mathcal{G} \}, \quad \mathcal{G} = \{ -180° + k \cdot 1°, k = 0 \ldots 359 \} \tag{39}$$

Le coût est de 360 rotations de vecteur par cible, soit une opération négligeable devant un seul appel de cinématique inverse. La précision est en revanche bornée à ±0,5°, ce qui est suffisant puisque cette valeur ne sert que de **graine** à un balayage ultérieur de pas 5°.

7.3 Solution analytique

Le balayage peut être remplacé par une expression fermée, ce qui constitue une piste d'optimisation immédiate. Le point de départ est une observation géométrique : lorsque le mandrin tourne, la normale de la bushing conserve un angle constant avec l'axe de rotation, si bien que sa pointe décrit un cercle — la normale balaie un cône.



7.3.1 Séparer ce qui tourne de ce qui ne tourne pas

On décompose n_0 en une composante le long de l'axe et une composante perpendiculaire :

$$n_0 = (n_0 \cdot a)\, a + n\perp\,, \quad n\perp = n_0 - (n_0 \cdot a)\, a \tag{40}$$

Une rotation autour de a laisse invariante la composante axiale ; seule $n\perp$ tourne, et elle tourne dans le plan perpendiculaire à l'axe. Dans ce plan, les deux vecteurs $n\perp$ et $a \times n\perp$ forment une base orthogonale de même norme — a étant unitaire et perpendiculaire à $n\perp$. Tourner d'un angle τ revient donc simplement à prendre les coordonnées polaires $(\cos\tau, \sin\tau)$ dans cette base :

$$n(\tau) = (n_0 \cdot a)\, a + \cos\tau \cdot n\perp + \sin\tau \cdot (a \times n\perp) \tag{41}$$

C'est la formule de Rodrigues du §4.2, mais réécrite de façon à faire apparaître explicitement la dépendance en τ : elle est purement sinusoïdale.

7.3.2 Projeter, puis réduire à un seul cosinus

Le produit scalaire étant linéaire, il traverse la somme sans difficulté. En projetant sur l'axe de l'outil :

$$g(\tau) = n(\tau) \cdot z_t = c + A\cos\tau + B\sin\tau \tag{42}$$

$$c = (n_0 \cdot a)(a \cdot z_t)\,, \quad A = n\perp \cdot z_t\,, \quad B = (a \times n\perp) \cdot z_t \tag{43}$$

Les trois quantités c, A, B sont des **constantes** : elles ne dépendent que de la géométrie du montage, pas de τ. La fonction à optimiser est donc une simple sinusoïde décalée verticalement. L'identité d'addition harmonique permet de la ramener à un unique cosinus déphasé :

$$g(\tau) = c + \sqrt{(A^2 + B^2)} \cdot \cos(\tau - \varphi)\,, \quad \varphi = \operatorname{atan2}(B, A) \tag{44}$$

— identité qui se vérifie en développant le membre de droite : $\sqrt{(A^2+B^2)}\cos\varphi = A$ et $\sqrt{(A^2+B^2)}\sin\varphi = B$ par définition même de φ.

Maximiser $f = -g$ revient à minimiser g, donc à placer le cosinus à sa valeur -1 :

$$\tau^* = \varphi + 180^\circ \pmod{360^\circ} \quad (45)$$

Géométriquement, φ est l'angle de mandrin pour lequel la normale pointe **le plus vers** l'outil ; un demi-tour plus loin, elle pointe le plus à l'opposé — ce que l'on recherche.

7.3.3 Critère d'atteignabilité de l'alignement

La forme fermée fournit en outre un diagnostic que le balayage discret ne donne pas. En notant $\beta = \angle(n_0, a)$ l'ouverture du cône et $\gamma = \angle(z_t, a)$ l'inclinaison de l'axe de l'outil (figure 1), un calcul direct donne :

$$c = \cos\beta \cdot \cos\gamma\,, \qquad \sqrt{(A^2 + B^2)} = \sin\beta \cdot \sin\gamma \quad (46)$$

d'où, par la formule d'addition des cosinus, la valeur minimale atteignable :

$$\min_\tau\ g(\tau) = \cos\beta\cos\gamma - \sin\beta\sin\gamma = \cos(\beta + \gamma) \quad (47)$$

L'alignement antiparallèle parfait, qui demande $g = -1$, est donc atteignable **si et seulement si** :

$$\beta + \gamma = 180^\circ \quad (48)$$

Autrement dit : il faut que le cône décrit par la normale **contienne** la direction $-z_t$. Dans le cas contraire, aucun angle de mandrin ne présentera la bushing parfaitement face à l'outil, et l'écart résiduel vaut exactement $|\,180^\circ - (\beta + \gamma)\,|$.

> **Valeur diagnostique**
> Ce critère sépare nettement les deux causes possibles d'un mauvais alignement. L'angle β est imposé par la géométrie de la pièce et l'orientation de l'axe du mandrin ; l'angle γ dépend uniquement de la pose de repos `iHomeJoints`. Si $\beta + \gamma$ s'écarte de 180°, c'est donc γ — donc la pose de repos — qu'il faut corriger, et non les paramètres de balayage du mandrin. Le balayage discret, lui, se contente de renvoyer un produit scalaire médiocre sans indiquer d'où vient le problème.

Sur le plan du coût, la forme fermée est exacte — sans erreur de discrétisation — et ne demande que quelques produits scalaires et deux appels à $atan2$, contre 360 rotations de vecteur pour le balayage.

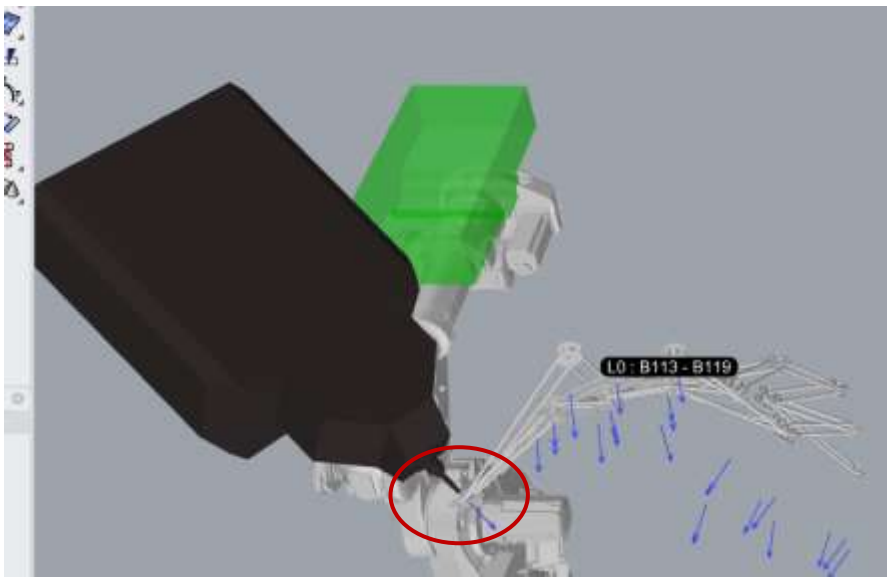
7.4 Génération des candidats de mandrin

Autour de la graine θ_s, les angles testés forment une spirale bornée par une fenêtre W et de pas h :

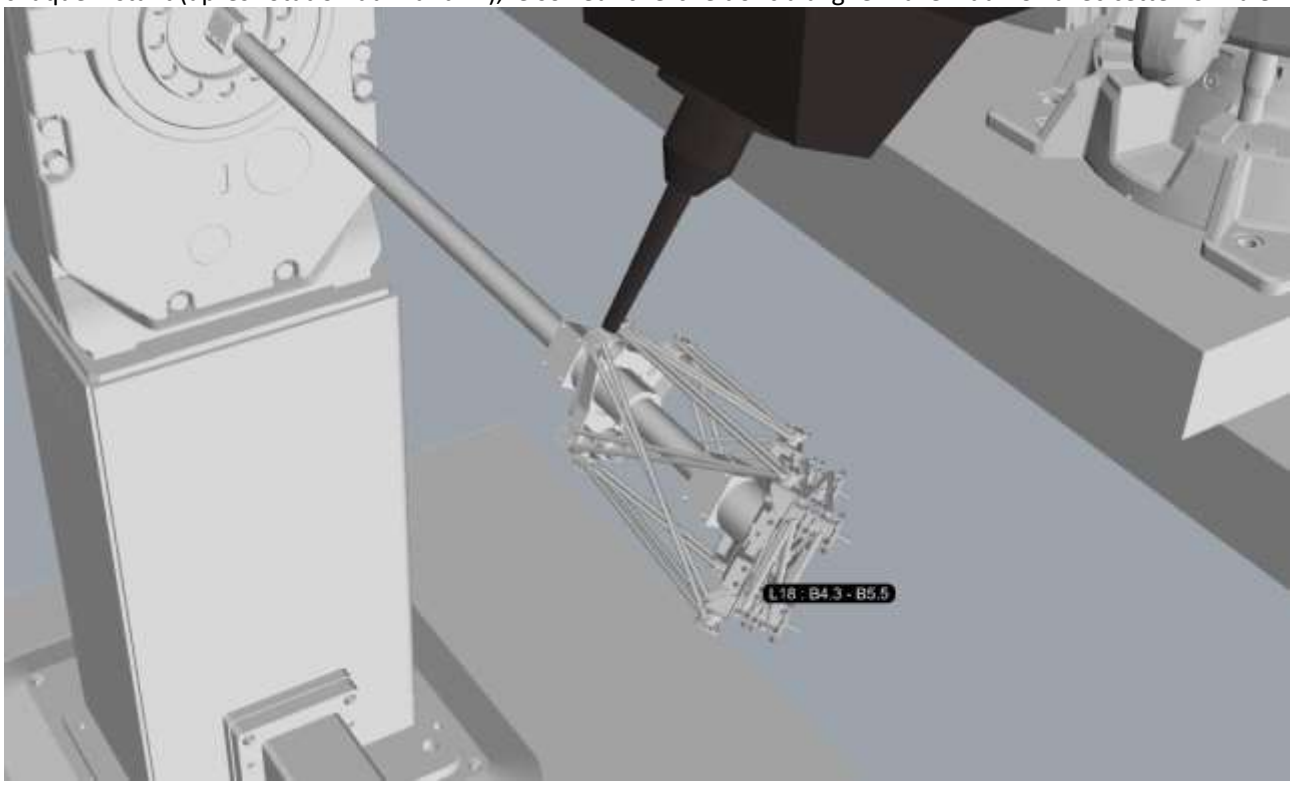
$$\mathcal{C} = (\,\theta_s\,,\ \theta_s + h\,,\ \theta_s - h\,,\ \theta_s + 2h\,,\ \theta_s - 2h\,,\ \ldots\,) \cap [\,\theta_s - W\,,\ \theta_s + W\,] \quad (46)$$

avec doublons éliminés (arrondi à 10^{-3}) et saturation aux bornes. Le cardinal est $|\mathcal{C}| = 2\lfloor W/h \rfloor + 1$, soit 13 candidats pour les valeurs par défaut $W = 30^\circ$, $h = 5^\circ$.

> **Stratégie « centrée » plutôt que « gloutonne »**
> Une variante antérieure ancrait la recherche de chaque bushing sur l'angle retenu pour la **bushing précédente** (stratégie gloutonne), afin de minimiser la rotation du mandrin entre deux points. Elle a été abandonnée au profit de la stratégie **centrée**, où chaque bushing repart de son propre optimum θ_s. Le coût est une rotation de mandrin potentiellement plus importante entre bushings ; le bénéfice est que la normale de chaque bushing est correctement réalignée sur l'outil, ce qui donne des postures robot nettement plus confortables et un taux de résolution supérieur. L'ancrage glouton propageait au contraire les compromis d'une bushing sur toutes les suivantes.



Simulation de la pièce Novacom2 avec le mandrin, en bleu nous pouvoir les vecteurs normaux de nos bushing à chaque instant (après rotation du mandrin), le solveur cherche donc à aligner l'axe Z du TCP avec cette normale.



Simulation des positions et de la rotation du mandrin pour une pièce du satellite Goffset.

8. Algorithme de recherche

8.1 Prédicat d'acceptation

Un triplet candidat (θ_b, E_1, ψ) est accepté si et seulement si les quatre conditions suivantes sont réunies :

$$Accept(\theta_b, E_1, \psi) = [\,\mathcal{E} = \emptyset\,] \wedge [\,IsInLimits\,] \wedge [\,q \in \mathcal{Q}\,] \wedge [\,\neg\, Collision\,] \quad (47)$$

où $\mathcal{E}$ est l'ensemble des messages d'erreur retournés par la cinématique inverse (hors d'atteinte, singularité, configuration impossible), $IsInLimits$ le verdict interne de RobotComponents, et $\mathcal{Q}$ le pavé des butées **de projet** — volontairement plus serré que les butées du constructeur :

$$q \in \mathcal{Q} \iff \forall k \in \{1..6\},\ \ q_k\text{^}min \le q_k \le q_k\text{^}max \quad (48)$$

Les deux tests de butées sont redondants en apparence mais pas en pratique : le premier applique les limites du modèle de robot chargé, le second des limites resserrées par l'utilisateur pour interdire des postures physiquement légales mais indésirables (proximité d'une singularité de poignet, câblage contraint, etc.).

8.2 Structure des boucles

Le parcours est un triple emboîtement, ordonné du paramètre le plus contraint au plus libre :

```
pour chaque cible i :
    plan monde de référence   P_ref  ← Stabilize( T_ref · P_i )
    E1_base ← ComputeE1(P_ref) ;  ψ*_base ← IdealYaw(P_ref, E1_base)
    graine mandrin θ_s ← Theta(n₀, z_t)  si iUseTheta,  sinon iEjbSeed

    pour chaque θ_b ∈ 𝒞(θ_s, W, h) :                    ← 13 valeurs
        T_ejb   ← T_u(θ_b)
        P_ejb   ← Stabilize( T_ejb · P_i )
        E1_base ← ComputeE1(P_ejb) ;  ψ* ← IdealYaw(P_ejb, E1_base)

        pour chaque ΔE1 ∈ [ E1_prev−E1_base , 0 , 𝒮(200,1400) ] :   ← 16 valeurs
            E1 ← E1_base + ΔE1
            si E1 ∉ [0, 9000] : continuer

            pour chaque Δψ ∈ [ −ψ* ] ⌢ 𝒮(YAW_STEP, YAW_MAX_RANGE) :
                si essais_ejb ≥ iMaxIkPerEjb : sortir            ← 12 max
                si essais_total > iMaxAttempts : abandonner la cible

                ψ      ← ψ* + Δψ
                P_fix  ← FixForRobot( Rotate( P_ejb , ψ ) )
                ext    ← ( E1 , θ_b , 9E9, 9E9, 9E9, 9E9 )
                q, ℰ   ← IK( P_fix , ext , outil , wobj_identité )

                si HorsAtteinte(ℰ) :
                    série++ ; si série ≥ iOutOfReachTol : sortir (θ_b suivant)
                    continuer
                série ← 0

                mémoriser (q, E1, θ_b, ψ) comme repli

                si ℰ = ∅ et q ∈ 𝒬 :
                    si Collision(q, E1, θ_b) : continuer   (yaw suivant)
                    ACCEPTER et sortir des trois boucles
```

8.3 Court-circuit « hors d'atteinte »

Le mécanisme piloté par `iOutOfReachTol` mérite une justification géométrique. La rotation de yaw **ne modifie pas la position** du TCP, seulement son orientation :

$$\| \Phi(P, \psi).O - B \| \text{ indépendant de } \psi \quad (49)$$

Or un diagnostic « hors d'atteinte » traduit essentiellement un problème de **distance**, non d'orientation. Si trois yaws consécutifs renvoient ce diagnostic, il est donc quasi certain que tous les autres le feront également : poursuivre la spirale serait du calcul pur perte. Le solveur sort immédiatement de la boucle de yaw et passe à l'angle de mandrin suivant, qui lui déplacera réellement la cible.

Le seuil de trois — plutôt que un — laisse une marge pour les cas où l'orientation influe malgré tout sur l'atteignabilité, typiquement à proximité immédiate de la limite d'enveloppe, où la longueur de l'outil projetée varie avec la pose du poignet.

8.4 Complexité

Le nombre d'appels de cinématique inverse par cible est majoré par :

$$N \leq \min(\ |\mathcal{C}| \cdot |\mathcal{E}_{E_1}| \cdot \min(|\Psi|, iMaxIkPerEjb)\ ,\ iMaxAttempts\ ) \quad (50)$$

Avec les valeurs par défaut, $|\mathcal{C}| = 13$, $|\mathcal{E}_{E_1}| = 16$ et $iMaxIkPerEjb = 12$, la borne combinatoire vaut $13 \times 16 \times 12 = 2496$, mais le plafond global `iMaxAttempts` (400 par défaut) est atteint bien avant.

La sortie `oAttempts` est donc à lire comme un **indicateur de marge** et non comme une simple mesure de temps de calcul.

8.5 Solution de repli

À chaque essai produisant une configuration articulaire non nulle — même hors butées — le solveur mémorise celle-ci comme **repli**. Si aucun candidat n'est finalement accepté, le repli est utilisé pour l'affichage, et le drapeau `oIsFallback` est levé. Cette configuration n'est **pas** exportée vers le code RAPID : elle sert uniquement au diagnostic visuel, en montrant à l'opérateur la pose la plus proche de l'admissibilité, ce qui rend immédiatement lisible la nature du blocage (bras replié, poignet retourné, cible franchement hors enveloppe).

9. Détection de collision outil / boîtier

Le modèle de robot fourni par RobotComponents ne comporte pas le boîtier électrique monté sur l'avant-bras. Or, pour un outil d'enroulement long, ce boîtier est l'obstacle le plus fréquemment rencontré. Le solveur intègre donc un maillage additionnel et teste explicitement l'interférence outil / boîtier avant d'accepter une pose.



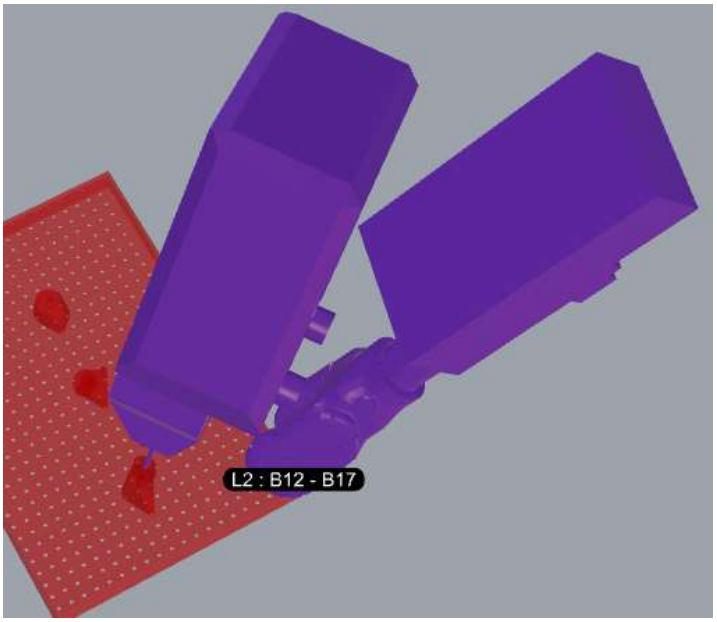
9.1 Repositionnement automatique entre robots

Le maillage du boîtier a été relevé sur un robot de calibration (TERRA), donc exprimé dans le repère monde de ce robot, de plan de position P_calib. Les cinq robots de la cellule étant du même modèle, le boîtier occupe la même position **locale** relativement à la base. Il suffit donc d'une transformation de repère à repère :

$$T_reloc = PlaneToPlane(P_calib, P_robot) = T_robot \cdot T_calib^{-1} \quad (51)$$

$$boîtier_courant = T_reloc \cdot boîtier_calib \quad (52)$$

Pour le robot de calibration lui-même, $P_calib = P_robot$ et $T_reloc = I$: l'opération est neutre. Ce mécanisme évite d'avoir à recaler manuellement le maillage pour chacun des cinq robots.



9.2 Recalage rigide par paire de maillages

Le boîtier doit suivre rigidement l'avant-bras $J3$ au fil des poses. Le problème posé est : connaissant le maillage de $J3$ à la pose de référence et à la pose courante, retrouver la transformation rigide qui relie les deux. Comme il

s'agit du **même corps rigide**, avec les mêmes sommets dans le même ordre, le recalage est **exact** et ne nécessite que trois points non colinéaires.

Le code procède ainsi :

4. Calcul des centroïdes : $c = (1/n)\,\Sigma_i\, v_i$ pour les deux maillages.
5. Choix du premier point : le sommet le plus éloigné du centroïde de référence, $i_a = arg\,max_i \,\|\, v_i - c \,\|^2$, ce qui maximise le bras de levier angulaire.
6. Choix du second point : celui qui maximise l'aire du triangle formé avec les deux premiers, $i_b = arg\,max_i \,\|\, (v_\{i_a\} - c) \times (v_i - c) \,\|^2$. Ce critère **garantit la non-colinéarité** — un produit vectoriel nul signifierait des points alignés, donc un repère dégénéré.
7. Construction des deux repères et de la transformation :

$$F_ref = Plane(\, c_ref,\, v_\{i_a\}\text{^}ref - c_ref,\, v_\{i_b\}\text{^}ref - c_ref\,) \quad (53)$$

$$T = PlaneToPlane(\, F_ref,\, F_cur\,) = T_cur \cdot T_ref^{-1} \quad (54)$$

Ce procédé est une version dégénérée mais exacte de l'algorithme de Kabsch : le recalage par moindres carrés sur n points n'apporterait rien puisque le résidu est nul par construction, alors qu'il coûterait une décomposition en valeurs singulières à chaque essai.

9.3 Test d'interférence en deux phases

L'intersection maillage-maillage est coûteuse ; elle est donc précédée d'un rejet rapide par boîtes englobantes :

$$Phase\ 1 :\ \ BB(A) \cap BB(B) = \emptyset \ \ \Longrightarrow\ \ pas\ de\ collision \quad (55)$$

$$Phase\ 2 :\ \ \mathcal{L} = MeshMeshFast(A, B)\,,\ \ Collision \Leftrightarrow |\,\mathcal{L}\,| > 0 \quad (56)$$

où $\mathcal{L}$ est l'ensemble des segments d'intersection entre les deux surfaces. Les boîtes englobantes étant alignées sur les axes du monde, le test de la phase 1 se réduit à trois comparaisons d'intervalles et élimine la grande majorité des configurations pour un coût négligeable.

> **Traitement d'une collision détectée**
> Une pose en collision n'est **pas** une pose invalide au sens de la cinématique : elle est simplement rejetée, et la spirale de yaw **se poursuit** à la recherche d'une orientation d'outil qui dégage le boîtier. La position du TCP étant inchangée par le yaw, c'est bien le levier pertinent. Le drapeau `oCollision` signale a posteriori qu'au moins une pose par ailleurs valide a été écartée pour cette raison.

10. Interface du composant

10.1 Principales entrées

Entrée	Défaut	Rôle mathématique
iPlanesWobj	—	plans cibles P_i en coordonnées pièce
iRob	—	modèle du robot (FK, IK, butées constructeur, plan de base)
joint_limits	—	définit le pavé $\mathcal{Q}$ (six intervalles)
flipOffsetDirection	bool	signe ε du déport ; côté d'implantation du robot
Y_min / Y_max	—	bornes de l'interpolation du déport
offset_min / offset_max	—	valeurs d_min et d_max du déport latéral
iAxisStr / iOframeStr	—	poses T_a et T_o du workobject sur axe externe

Entrée	Défaut	Rôle mathématique
iTeachingAngle	—	angle mandrin θ_teach de l'apprentissage
iMandrelAxis	z	indice k de la colonne donnant la direction a
iUseTheta	false	active le calcul analytique de la graine θ_s
iEjbSeed	−30	graine constante lorsque iUseTheta est désactivé
iEjbWindow / iEjbStep	30 / 5	fenêtre W et pas h du balayage de mandrin
YAW_STEP / YAW_MAX_RANGE	—	pas et amplitude de la spirale de yaw
iHomeJoints	0,0,0,0,30,0	pose de repos q_home définissant z_t
iJ4Box / iCalibPP	—	maillage du boîtier et repère de calibration
iMaxIkPerEjb	12	plafond d'essais IK par angle de mandrin
iOutOfReachTol	3	seuil du court-circuit hors d'atteinte
iMaxAttempts	400	plafond global d'essais par cible

10.2 Sorties de diagnostic

Les sorties permettent de vérifier **numériquement** — et non visuellement — que la chaîne a produit le résultat attendu. C'est la méthode de validation retenue tout au long du développement : une inspection visuelle ne détecte pas une erreur de 180° compensée, ni un décalage d'arborescence de données.

Sortie	Lecture
oSolved	booléen par cible ; premier indicateur à consulter
oAttempts	nombre d'essais IK ; mesure la marge de la solution
oE1_base / oE1_final	écart entre le rail idéal et le rail retenu
oYaw_base / oYaw_final	dérive du yaw par rapport à l'idéal ; un écart important signale un mauvais point de départ
oEJb_final	angle de mandrin retenu (absolu)
oIsFallback	vrai si aucune solution admissible n'a été trouvée
oCollision	vrai si au moins une pose valide a été rejetée pour interférence
oPlaneSrc / oPlaneTarget	plan brut et plan après la chaîne Φ ; vérification visuelle du retournement
debug	trace complète par cible : theta, plage de mandrin, court-circuits, acceptation

11. De la pose au robtarget : finalité du solveur

Tout ce qui précède n'a de sens qu'au regard de son aboutissement : la production d'un module RAPID exécutable par le contrôleur ABB. Cette section explique la traduction — purement formelle — entre la solution géométrique du solveur et la déclaration `robtarget` qui la porte jusqu'au robot.

11.1 Structure du robtarget

Le `robtarget` est le type de donnée fondamental du langage RAPID pour désigner une cible : toutes les instructions de mouvement, `MoveL` comme `MoveJ`, en prennent un en argument. Sa structure comporte quatre blocs :

```
VAR robtarget <nom> := [ [ x, y, z ],
                         [ q1, q2, q3, q4 ],
                         [ cf1, cf4, cf6, cfx ],
                         [ eax_a, eax_b, eax_c, eax_d, eax_e, eax_f ] ];
```

Ces quatre blocs correspondent **exactement** aux quatre décisions prises par le solveur pour chaque cible. C'est cette correspondance terme à terme qui justifie la structure de l'algorithme :

Bloc RAPID	Contenu	Produit par
trans — [x, y, z]	origine du plan cible, en millimètres	la géométrie d'enroulement, inchangée par le solveur
rot — [q1…q4]	quaternion de l'orientation, scalaire en tête	la chaîne d'orientation Φ du §5, incluant le yaw ψ
robconf — [cf1, cf4, cf6, cfx]	famille de posture parmi les huit (§2.3.1)	imposé : ConfigurationData(1, 0, 0, 0)
extax — [eax_a … eax_f]	positions des six axes externes	eax_a = E_1 (rail), eax_b = θ_b (mandrin), reste à 9E9

La valeur conventionnelle `9E9` signale au contrôleur qu'un axe externe n'est **pas défini** pour cette cible et doit être ignoré. On la retrouve dans le code du solveur à chaque construction d'un `ExternalJointPosition`, où seules les deux premières composantes portent une valeur réelle.

Une déclaration réelle produite par la chaîne, telle qu'elle figure dans le manuel opérateur, prend la forme :

```
VAR robtarget Task_101_1_0 :=
    [[787.38, 514.93, 53.01],
     [0.081038, -0.117485, 0.988146, 0.056556],
     [1, 0, 0, 0],
     [5950, 9E9, 9E9, 9E9, 9E9, 9E9]];
```

On y lit directement le travail du solveur : le rail est placé à 5950 mm, la configuration est bien celle imposée, et le quaternion encode l'orientation issue de la chaîne Φ. Ici le mandrin n'est pas utilisé (`eax_b = 9E9`) ; dans la configuration avec mandrin, cette case porte la valeur θ_b retenue.

11.2 Extraction des sept nombres

Le solveur ne manipule pas des nombres mais des repères. Le passage du plan retenu $P_fix = \Phi(\mathcal{T}(P_i, \theta_b), \psi)$ aux sept scalaires du robtarget est immédiat :

$$(x, y, z) = O(P_fix) \quad (61)$$

$$(q1, q2, q3, q4) = quat([\hat{x}\ \hat{y}\ \hat{z}]) \quad \text{par la méthode de Shepperd du §3.2.1} \quad (62)$$

La matrice dont on extrait le quaternion est celle dont les colonnes sont les trois axes du plan — c'est-à-dire le bloc rotation R de la matrice homogène du §3.1. La convention scalaire en tête est celle d'ABB, et elle est respectée de bout en bout dans la chaîne.

Ambiguïté de signe du quaternion

Les quaternions q et $-q$ représentent la **même** rotation : le groupe des quaternions unitaires est un revêtement double de $SO(3)$. Le contrôleur les interprète donc de façon identique, et l'ambiguïté est sans conséquence pour l'exécution. Elle en a une pour le **diagnostic** : deux générations successives du même programme peuvent produire des lignes textuellement différentes alors qu'elles sont géométriquement identiques. Comparer deux modules RAPID par différence de texte est donc trompeur — il faut comparer les rotations, ou normaliser le signe (par exemple en imposant $q1 \geq 0$) avant l'écriture.

11.3 Le rôle du workobject

Une même cible physique admet deux écritures équivalentes, reliées par la transformation du workobject :

$$X_monde = T_wobj \cdot X_local \quad \Leftrightarrow \quad X_local = T_wobj^{-1} \cdot X_monde \quad (63)$$

Le solveur travaille en interne avec un workobject **identité** — la ligne `new WorkObject("identity", Plane.WorldXY)` — et exprime donc ses cibles directement en coordonnées monde, l'effet du mandrin ayant déjà été appliqué géométriquement par $T_u(\theta_b)$. Les deux écritures possibles pour le module généré n'ont pourtant pas le même comportement à l'exécution :

Écriture	Coordonnées écrites	Comportement du contrôleur
wobj = identité	coordonnées monde, figées à l'angle θ_b du calcul	la cible ne suit pas le mandrin ; la valeur eax_b doit correspondre exactement à celle du calcul. Cette étape est essentiel pour simuler les positions sur le logiciel mais n'est jamais implanté dans le contrôleur.
wobj sur unité mécanique	coordonnées pièce, constantes	le contrôleur recalcule T_wobj à chaque instant depuis la position réelle du mandrin ; la cible suit la pièce

La seconde écriture est celle qui autorise le **mouvement coordonné** : le contrôleur planifie une trajectoire unique pour le bras et les axes externes, de sorte que le TCP suive un segment rigoureusement rectiligne dans le repère de la pièce pendant que celle-ci tourne.

11.4 Condition de cohérence

Le contrôleur ne reçoit pas les angles articulaires : il reçoit une **pose**, et résout lui-même la cinématique inverse au moment de l'exécution. La validité du programme repose donc sur une condition d'aller-retour :

$$FK_contrôleur(\, IK_contrôleur(\, X\,)\,) = X = FK_solveur(\, q, E_1, \theta_b\,) \quad (64)$$

Par construction, le solveur garantit le membre de droite : il n'accepte un candidat que si l'IK a convergé sans erreur. Mais l'égalité entre les deux IK — celle de RobotComponents et celle du contrôleur — n'est acquise que si **quatre données sont rigoureusement identiques** de part et d'autre :

1. **Le modèle de robot et l'outil** : le `tooldata` du contrôleur doit reproduire exactement le TCP et la masse déclarés dans Grasshopper. Un décalage de TCP se traduit directement en décalage de trajectoire.
2. **Le workobject** : la chaîne `wobjdata` doit correspondre à celle sous laquelle les cibles ont été écrites (§11.3).
3. **La configuration** : le `confdata` écrit doit être celui sous lequel la solution a été trouvée, faute de quoi le contrôleur choisira une autre des huit solutions.
4. **Les axes externes** : E_1 et θ_b doivent être les valeurs exactes du calcul, puisque la pose du TCP dans le monde en dépend.

Ces quatre points constituent la surface de contact entre la simulation et la cellule réelle. Une divergence sur l'un quelconque d'entre eux produit un programme syntaxiquement valide, simulé correctement, et pourtant faux à l'exécution — c'est le mode de défaillance le plus coûteux de la chaîne.

11.5 Ce que le robtarget ne contient pas

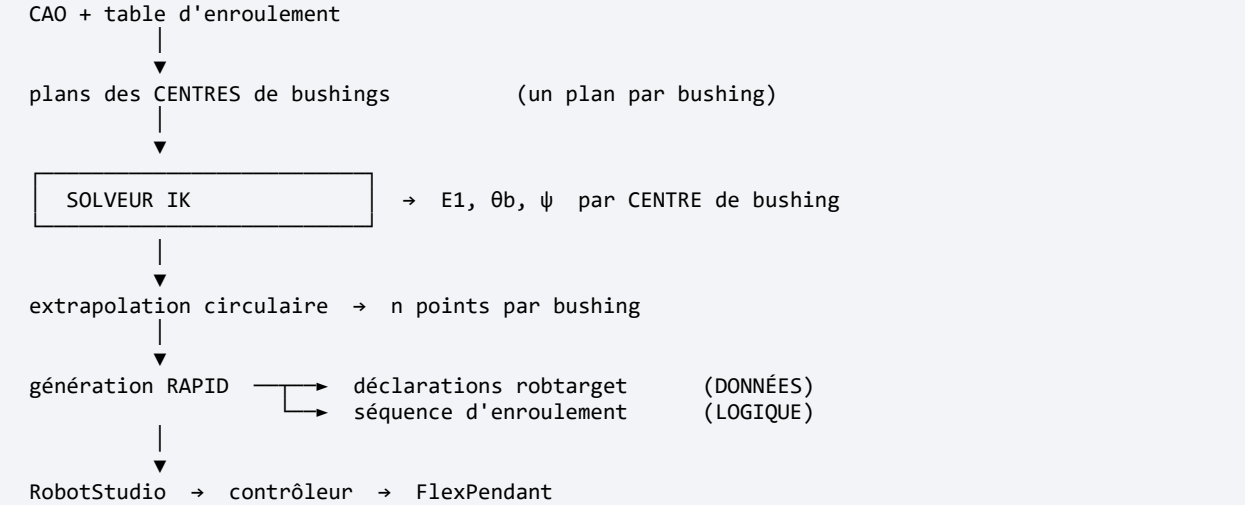
Un robtarget décrit un **point**, pas un **mouvement**. Trois informations essentielles lui sont extérieures :

- **Le type d'interpolation** — `MoveL` (trajectoire rectiligne dans l'espace opérationnel) ou `MoveJ` (interpolation dans l'espace articulaire). Le solveur raisonne en `MoveL`, cohérent avec un procédé de dépose continue.
- **La vitesse** — le `speeddata`, qui comporte quatre composantes : vitesse du TCP, vitesse d'orientation, vitesse des axes externes linéaires et vitesse des axes externes rotatifs.
- **La précision de passage** — le `zonedata`, qui autorise le contrôleur à arrondir le passage par la cible au lieu de s'y arrêter.

> **Une conséquence directe de la stratégie EJb centrée**
>
> En mouvement coordonné, le contrôleur synchronise le bras et les axes externes pour qu'ils **terminent ensemble**. Si le TCP ne parcourt qu'une faible distance alors que le mandrin doit tourner de plusieurs dizaines de degrés, l'axe rotatif accélère fortement pour tenir la synchronisation. Or c'est précisément le comportement que produit la stratégie « centrée » du §7.4, qui privilégie le confort de posture au prix de rotations de mandrin plus amples entre bushings. Le remède se situe en aval du solveur : limiter la composante v_reax du `speeddata`, voire réduire l'accélération globale par `AccSet` pour un montage à forte inertie. C'est un bon exemple de couplage entre un choix d'algorithme et un réglage de contrôleur.

11.6 Place dans la chaîne de production

Le solveur occupe une position centrale mais délimitée dans le flux de travail décrit par les instructions d'exploitation :



11.6.1 Les blocs logiques de la définition Grasshopper

Les instructions d'exploitation recensent les fonctions qui constituent le cœur logique des séquences d'enroulement, situées au milieu de la définition Grasshopper. Le solveur n'en est qu'un maillon :

Bloc	Rôle	Relation au solveur
Z-plane offset / Strategy	placement vertical de chaque point d'enroulement sur la hauteur de la bushing	amont — fixe la position des cibles
Solver determination	résolution des poses aux centres de bushings	le présent document
Rotate Planes	orientation des plans cibles	amont immédiat — alimente la chaîne Φ
Robot Targets	extrapolation autour des centres puis écriture des robtarget	aval direct — §1.4 et §11.1 à §11.3
Winding Syntax	ordre de dépose des fibres	aval — définit la séquence des cibles

Bloc	Rôle	Relation au solveur
Validation par visualisation	contrôle du mouvement du robot	consomme les sorties d'affichage du solveur
Robot position	implantation du robot dans la cellule	amont — fixe le plan de base et le côté d'implantation

Cette décomposition éclaire une propriété importante du solveur : il est **sans mémoire de procédé**. Il ne sait ni pourquoi une cible se trouve à cette hauteur, ni dans quel ordre elle sera visitée. Il reçoit une liste de plans et rend une liste de configurations, cible par cible.

11.6.2 En amont — construction des cibles autour d'un bushing (déjà implanté auparavant)

Chaque bushing est décrite par un nombre de cibles adapté à sa géométrie. Pour un bushing circulaire standard, la construction est régulière : on définit un **plan de référence central** qui coupe la bushing à mi-hauteur, puis on engendre une série de plans **parallèles** à celui-ci, répartis sur la hauteur. Chaque plan porte ensuite ses propres points cibles, disposés autour du bushing, chacun avec son repère local.

C'est le **plan de référence central** — et lui seul — qui est transmis au solveur. Les points répartis autour de la douille en sont déduits par l'extrapolation circulaire du §1.4, une fois la configuration robot établie. Le solveur voit donc **une entrée par bushing**, là où le module RAPID final comportera n déclarations `robtarget` par bushing.

Deux propriétés de cette construction ont des conséquences directes sur l'algorithme :

- **L'orientation de l'outil reste colinéaire à la normale du plan cible** tout au long du parcours. C'est ce qui rend la rotation de yaw ψ légitime : elle laisse la normale invariante (§5.2) et ne perturbe donc pas le procédé — et c'est aussi ce qui permet de la transporter du centre vers les points extrapolés.
- **L'outil parcourt les points séquentiellement** pour éviter les collisions avec les bushings. Cette contrainte de parcours est extérieure au solveur, qui ne raisonne jamais sur l'enchaînement des points d'une même douille.

Pour les bushings non circulaires, le nombre de cibles diffère afin d'assurer une couverture complète et précise de la fibre. Le solveur y est insensible : le nombre de points extrapolés ne change ni son entrée, ni son coût, ni son résultat.

11.6.3 En aval — la syntaxe d'enroulement

La **winding syntax** est le composant qui détermine l'**ordre** dans lequel les fibres sont déposées, avec pour objectif de produire un parcours sans collision. Elle obéit à une hiérarchie stricte :

- les **liens inférieurs** sont toujours enroulés avant les liens supérieurs ;
- les bushings empilées sont traitées séquentiellement **du bas vers le haut** ;
- sur une bushing donnée, tous les **liens internes** sont achevés en premier ;
- les **demi-cycles** sont évités.

Ces règles définissent un **ordre partiel** sur l'ensemble des liens — une relation de précédence — que la définition résout par tri topologique. Le résultat est une séquence linéaire compatible avec toutes les contraintes.

> **Séparation des responsabilités**
>
> Il y a ici une frontière nette, et il est utile de la formuler explicitement dans un rapport : la syntaxe d'enroulement traite des collisions **fibre / pièce** — l'ordre de dépose doit éviter qu'une fibre déjà posée ne fasse obstacle. Le solveur traite, lui, des collisions **outil / robot** (§9) et de l'atteignabilité mécanique. Les deux problèmes sont disjoints et résolus par des blocs différents. Aucun des deux ne peut compenser une défaillance de l'autre : un ordre d'enroulement correct n'aide en rien une cible hors d'atteinte, et une pose parfaitement confortable ne sauve pas une fibre déposée dans le mauvais ordre.

11.6.4 Séparation données / logique

Le module généré sépare volontairement les **données** — les positions cibles — de la **logique** — les procédures d'enroulement, organisées en une procédure principale, des procédures de lien et des procédures de séquence. Cette séparation a une conséquence pratique importante : lorsqu'un paramètre géométrique change (rotation d'un plan, position du rail, angle de mandrin), seules les déclarations de cibles doivent être régénérées ; la séquence d'enroulement reste valable tant que l'algorithme d'enroulement lui-même n'a pas changé.

Autrement dit, **le solveur produit exclusivement des données**. Il ne décide ni de l'ordre des mouvements, ni des stratégies d'enroulement, ni des transitions entre liens — ces éléments relèvent des blocs situés en aval. Sa responsabilité est strictement délimitée : garantir que chacune des cibles transmises est **atteignable, dans les butées et sans collision**, et que les valeurs d'axes externes qui l'accompagnent sont celles sous lesquelles cette garantie a été établie.

12. Discussion et perspectives

12.1 Nature de l'optimum obtenu

Le solveur est **glouton par centre** : il accepte le premier candidat admissible et ne remet jamais en cause un choix passé. La seule mémoire inter-centres est la préférence de continuité sur le rail. Formellement, la solution produite pour la séquence complète n'est donc pas :

$$\arg\min_{(s_i)} \Sigma_i \ \text{coût}(s_i) + \Sigma_i \ \text{transition}(s_{i-1}, s_i) \quad (67)$$

mais une solution admissible construite localement. La qualité du résultat dépend donc entièrement de la qualité de l'**ordre de parcours** : c'est pourquoi les paramètres qui déterminent le **point de départ** de la recherche — côté d'implantation, pose de repos, déport — ont un effet bien supérieur à ceux qui en fixent l'**étendue**.

12.2 Vers une optimisation globale

La structure du problème — une séquence de centres, un ensemble discret de configurations candidates par centre, un coût de transition entre configurations consécutives — est exactement celle d'un problème de plus court chemin dans un graphe en couches. Il se résout par **programmation dynamique** (algorithme de Viterbi) en :

$$O(N \cdot K^2) \quad N = \text{nombre de centres}, \ K = \text{candidats retenus par centre} \quad (68)$$

avec la récurrence de Bellman classique :

$$C_i(s) = \text{coût}(s) + \min_{s' \in \mathcal{K}_{i-1}} [\, C_{i-1}(s') + \text{transition}(s', s) \,] \quad (69)$$

Ce cadre permettrait de minimiser explicitement les déplacements du rail et du mandrin sur l'ensemble du cycle, et d'écarter les postures marginales au profit de postures confortables. Le coût est qu'il faut **énumérer** les K candidats de chaque centre avant de décider, là où le solveur actuel s'arrête au premier — soit un volume de cinématique inverse considérablement supérieur. Un compromis praticable consisterait à ne retenir que quelques candidats bien répartis par cible, en utilisant le solveur actuel comme générateur.

12.3 Améliorations ponctuelles identifiées

- **Angle theta en forme fermée** (§7.3) : suppression de 360 évaluations par centre et de l'erreur de discrétisation, plus un diagnostic d'atteignabilité de l'alignement.
- **Yaw idéal par atan2** (§6.3) : conditionnement numérique uniforme, contre une dégradation de $\arccos$ précisément pour les petits angles.
- **Zones de collision couplées** (§9.4) : modélisation des régions interdites en $(A4, A5, A6)$ plutôt que des butées indépendantes par axe.
- **Mise en cache de la cinématique directe de repos** : le TCP à la pose de repos ne dépend que de E_1 ; son calcul dans la boucle réinstancie l'ensemble des maillages du robot à chaque appel.
- **Recalage rigide hors boucle** : la transformation du boîtier ne dépend que de la configuration articulaire, elle pourrait être factorisée avec le calcul de cinématique directe déjà effectué.

12.4 Synthèse

Le solveur résout un problème de cinématique inverse redondante à trois degrés de liberté excédentaires, en le ramenant à une recherche ordonnée sur trois paramètres scalaires — angle de mandrin, position de rail, yaw d'outil. Sa robustesse repose sur trois choix :

1. un **placement déterministe** de chaque paramètre libre, calculé géométriquement (interpolation du déport, angle signé vers la base, alignement normale-outil), qui donne un point de départ proche de la solution ;

2. des **parcours en spirale** qui énumèrent les alternatives par distance croissante à ce point de départ, si bien que le premier candidat valide est également un bon candidat ;
3. une **chaîne d'orientation stricte** appliquée exactement une fois, qui garantit la cohérence entre la géométrie modélisée, la convention outil du robot et le workobject écrit dans le code RAPID.

Ses limites tiennent au caractère local de la décision : le solveur produit une solution admissible, non une solution optimale au sens du cycle complet. Le passage à une formulation par programmation dynamique constitue l'extension naturelle de ce travail.